

LEGALEASE

LEGALEASE:AI--POWERD LRGAL
DOCUMENT GENERATOR

ASAN MEMORIAL COLLEGE OF ARTS AND SCIENCE
BSC COMPUTER SCIENCE 2ND YEAR

TEAM MEMBERS:-

1)Z.MOHAMMED ARSHAD. (
NM ID):-

8FEBA3E6A8A9CDB9F0541301C4DDD389

2)ASHWIN RAJ.G(NM ID):- 006C32DE2C89676A3044
D790E4004C7D

3)MATHESH.S(NM ID):-
686472ABB089010A2BF46CA6BCEC9AF0

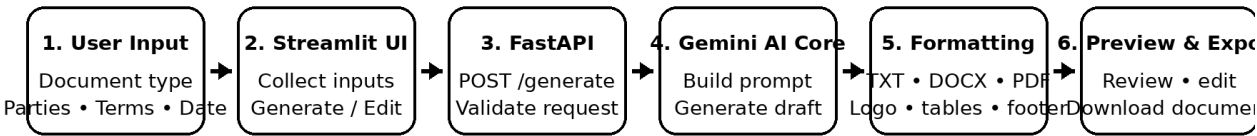
R)SAKTHIVEL(NM ID):-
8FA1C341A242290A7434109026D67EF5

PROJECT REPORT

LegalEase

AI-Powered Legal Document Generator

LegalEase — Project Workflow



Review & Validation Loop

Check required inputs → Review generated clauses → Edit content → Re-preview

If changes are needed, repeat preview/edit before final export.

Deployment: local testing → dependency verification → future cloud deployment

A structured project document based on the supplied LegalEase project PDF

Project Details	Entry
Project Title	LegalEase: AI-Powered Legal Document Generator
Technology	Python, FastAPI, Streamlit, Gemini API
Document Formats	.TXT, .DOCX, .PDF
Prepared By	_____
Register No.	_____
Institution / Department	_____
Academic Year	2026–2027

Certificate

This is to certify that the project report entitled “LegalEase: AI-Powered Legal Document Generator” is a bonafide project work prepared for academic/project purposes. The report presents the system design, workflow, implementation approach, API logic, user interface, document formatting, testing, and deployment procedure described in the supplied project material.

The work demonstrates the integration of a Streamlit frontend, FastAPI backend, Gemini-based generation component, and document export utilities. The implementation details in this report are organized from the supplied project document and expanded into a clear academic report format.

Project Guide: _____

Head of Department: _____

Date: _____ Place: _____

Declaration & Acknowledgement

Declaration

I declare that this project report, prepared from the supplied LegalEase project material, is presented for academic/project documentation. The report describes the intended architecture and implementation workflow of the system.

Acknowledgement

I express my sincere thanks to my project guide, faculty members, department, institution, and all resources that supported the preparation of this project report. I also acknowledge the supplied LegalEase project document as the primary source used to organize the technical content.

Important Scope Note

LegalEase is described as an AI-assisted legal-document generation system. Generated text should be treated as a draft for review rather than a substitute for advice from a qualified legal professional. The supplied source describes generation, editing, formatting, and export; it does not establish that every generated document is legally valid in every jurisdiction.

Abstract

LegalEase is an AI-powered legal document generation project designed to simplify the preparation of structured legal documents. The system accepts information such as document type, parties, terms and effective dates, sends the structured request through a FastAPI backend to a Gemini-based AI generation component, and returns editable document text. The project combines Streamlit for the user interface, FastAPI for backend API processing, Gemini for content generation, and Python libraries for document formatting and export. Users can preview generated content, edit it, and download the result as TXT, DOCX or PDF. The source project also describes branding support such as logos, headings, tables and footers. The report presents the complete project workflow, system architecture, requirements, installation commands, API syntax, core functions, user flow, testing procedure, deployment steps, advantages, limitations and conclusion.

Keywords

Artificial Intelligence, Generative AI, Gemini, FastAPI, Streamlit, Python, Legal Documents, DOCX, PDF, API

Table of Contents

1. 1. Introduction & Problem Statement
2. 2. Objectives and Scope
3. 3. Existing System and Proposed System
4. 4. Requirements and Technology Stack
5. 5. System Architecture
6. 6. Project Workflow
7. 7. Environment Setup
8. 8. Core Functionalities
9. 9. FastAPI Backend Implementation
10. 10. Gemini AI Integration
11. 11. Streamlit Frontend
12. 12. Document Formatting & Export
13. 13. User Operation / Project Work Flow
14. 14. Testing & Validation
15. 15. Deployment
16. 16. Advantages, Limitations & Future Enhancements
17. 17. Sample Code & Syntax Reference
18. 18. Conclusion
19. 19. References / Source Material

1. Introduction & Problem Statement

1.1 Introduction

Legal documents commonly require structured information, formal language and consistent formatting. The supplied LegalEase project proposes an AI-assisted workflow that converts user-provided details into a formatted legal-document draft.

1.2 Problem Statement

A user may know the basic facts of an agreement but may not know how to organize them into a formal document. The project addresses this usability problem by collecting structured inputs and generating a document draft that can be edited and exported.

1.3 Target Use Cases

- Employment contracts or offer documents

- Non-Disclosure Agreements (NDAs)

- Lease agreements

- General agreements and contracts

- Freelance or service-work contracts

1.4 Project Concept

The central concept is: Input → API processing → AI generation → Preview → Editing → Formatting → Export. This sequence is the foundation of the project workflow.

Source basis: the supplied project describes customizable legal templates, AI generation, editable previews, branding and export to PDF, DOCX and TXT. filecite turn0file0 L2-L14

2. Objectives and Scope

2.1 Objectives

- Build an easy-to-use interface for entering legal document details.
- Generate structured document drafts using a Gemini-based AI component.
- Validate and route requests through a FastAPI backend.
- Provide editable previews before download.
- Export generated content to TXT, DOCX and PDF.
- Apply formatting such as headings, terms tables, logos and footers.
- Provide a workflow that can be tested locally and prepared for deployment.

2.2 Scope

- Frontend: Streamlit user interface.
- Backend: FastAPI application and /generate endpoint.
- AI core: GeminiDocumentGenerator.
- Formatting: python-docx, FPDF and HTML preview.
- Configuration: environment variables such as the Gemini API key.
- Deployment: local deployment with possible cloud deployment later.

2.3 Inputs and Outputs

Input	Example	Output
Document type	Freelance Work Contract	Structured contract draft
Parties	Service Provider, Client	Party details in document
Terms	Payment; Confidentiality; Deadline	Clause list / terms table
Effective date	10/04/2025	Date included in document

3. Existing System and Proposed System

3.1 Existing / Manual Approach

- User searches for a suitable template.
- Relevant details are entered manually.
- Formatting is performed separately.
- Revisions may require repeated manual editing.
- Different output formats may require separate tools.

3.2 Proposed LegalEase Approach

- Collect structured inputs in one interface.
- Generate a draft using the AI core.
- Preview and edit the generated content.
- Automatically format the document.
- Download in multiple formats.

3.3 Comparison

Aspect	Manual Approach	LegalEase Workflow
Input	Template/manual entry	Structured UI
Drafting	Manual	AI -assisted
Editing	External/manual	Built-in preview/edit
Formatting	Manual	Automated utilities
Export	Separate tools	TXT/DOCX/PDF options
Workflow	Multiple steps/tools	Integrated flow

4. Requirements and Technology Stack

4.1 Software Prerequisites

Python 3.10+

pip package manager

FastAPI and Uvicorn

Streamlit

Google Generative AI SDK

python-docx

FPDF

Pillow

requests

python-dotenv

4.2 Installation Commands

```
pip install fastapi uvicorn
pip install streamlit
pip install google-generativeai
pip install python-docx fpdf Pillow requests python-dotenv
```

4.3 Technology Roles

Technology	Role
Python	Main programming language
Streamlit	Frontend and interactive UI
FastAPI	Backend REST API
Uvicorn	ASGI server for FastAPI
Gemini API	AI content generation
Pydantic	Request validation
python-docx	DOCX formatting
FPDF	PDF generation
Pillow	Image/logo handling
python-dotenv	Environment variable loading

The supplied project specifies these prerequisites and installation packages.

filecite turn0file0 L27-L33

5. System Architecture

LegalEase — System Architecture

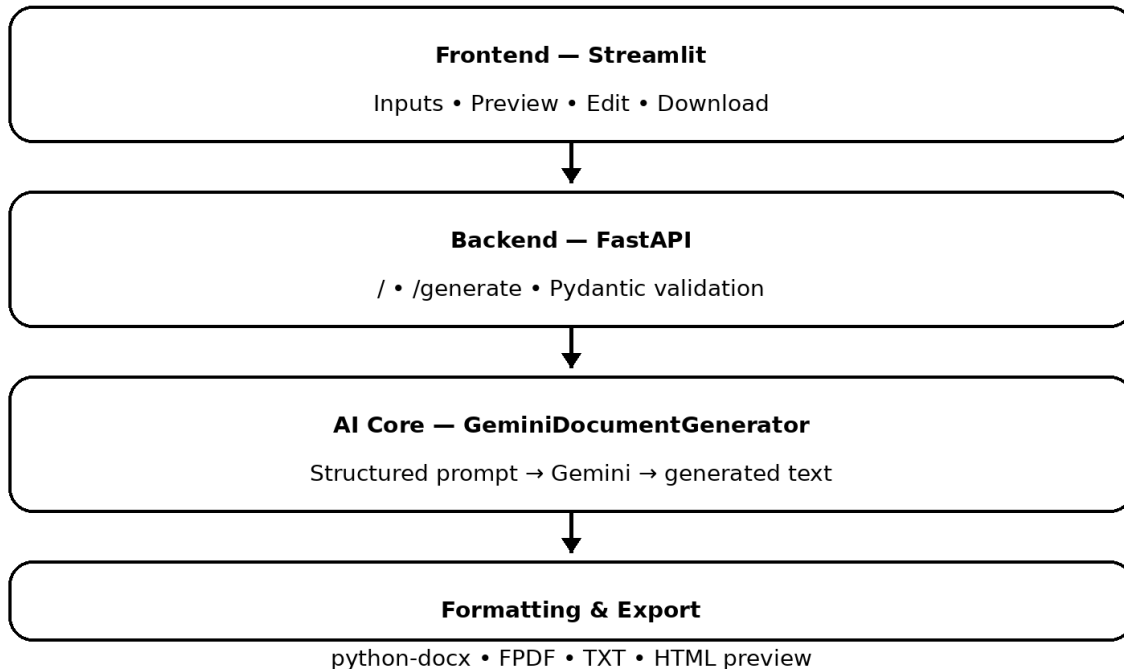


Figure 1. Logical architecture organized from the supplied project description.

5.1 Architecture Components

Frontend Layer: Streamlit collects document type, parties, terms and date; it also displays previews and download controls.

Backend Layer: FastAPI exposes API routes and validates incoming data using a Pydantic request model.

AI Layer: GeminiDocumentGenerator constructs a structured prompt and calls the Gemini generation service.

Formatting Layer: DOCX, PDF, TXT and HTML preview utilities transform the generated text into usable output.

The source explicitly identifies Streamlit, FastAPI, Gemini integration and DOCX/PDF/TXT formatting modules as the principal components. filecite turn0file0 L113-L121

6. Project Workflow

LegalEase — Project Workflow

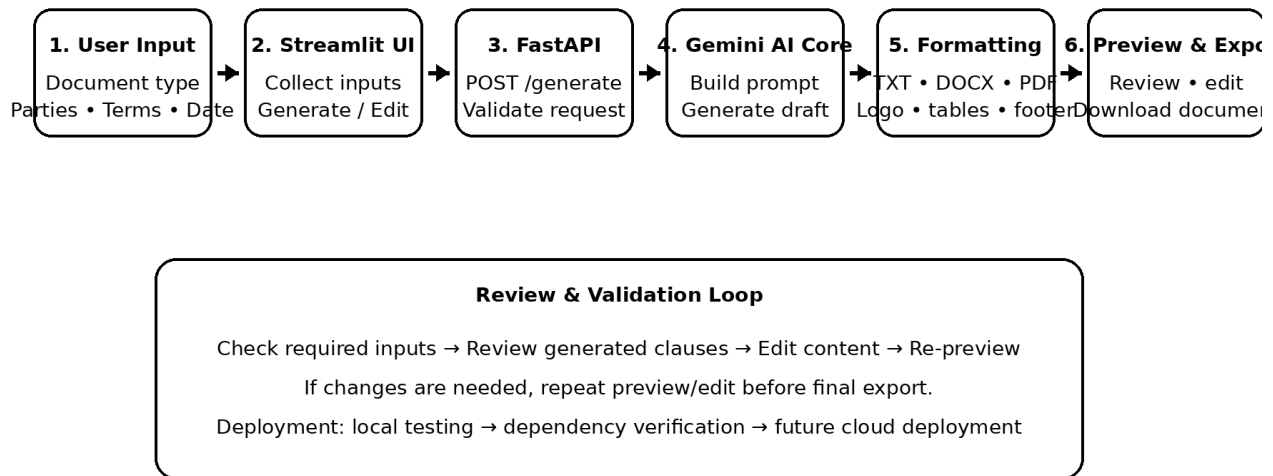


Figure 2. End-to-end LegalEase project workflow.

6.1 Step-by-Step Flow

1. User enters document type, parties, terms and effective date.
2. Streamlit collects the input and sends a POST request.
3. FastAPI validates the request and forwards it to the AI generator.
4. GeminiDocumentGenerator builds a structured prompt and obtains generated content.
5. The frontend displays a formatted preview.
6. User reviews or edits the generated content.
7. Formatting utilities create TXT, DOCX or PDF output.
8. User downloads the selected document.
9. Testing verifies layout, terms, logo/footer and output behavior.

7. Environment Setup

7.1 Create a Project Folder

```
mkdir LegalEase  
cd LegalEase
```

7.2 Create a Virtual Environment

```
python -m venv venv
```

```
# Windows  
venv\Scripts\activate
```

```
# Linux / macOS  
source venv/bin/activate
```

7.3 Install Dependencies

```
pip install -r requirements.txt
```

7.4 Environment Variable

Create a .env file and store the Gemini API key as an environment variable. The key should not be hard-coded into source files or shared publicly.

```
GEMINI_API_KEY=your_api_key_here
```

7.5 Suggested Project Structure

```
LegalEase/  
├── main.py  
├── routes.py  
├── app.py  
├── requirements.txt  
├── .env  
├── ai_core/  
│   └── gemini_generator.py  
└── assets/  
    └── logo.png
```

8. Core Functionalities

Functionality	Description
Legal document generation	Creates a document draft from structured user inputs.
Editable preview	Allows the generated content to be reviewed and modified.
TXT export	Downloads plain text.
DOCX export	Creates a formatted Word document.
PDF export	Creates a formatted PDF with branding.
HTML preview	Shows styled content inside the Streamlit UI.
Terms table	Converts semicolon-separated terms into a table.
Branding	Supports logos, headings and footers.

8.1 Core Utility Functions

`sanitize_text(text)`

`format_docx(text, doc_type)`

`format_pdf(text, doc_type)`

`format_html_preview(text)`

According to the supplied project, `sanitize_text()` removes problematic special characters/typographic quotes, `format_docx()` handles Word formatting and terms tables, `format_pdf()` handles PDF headers/footers and bullet-style terms, and `format_html_preview()` prepares styled HTML output. filecite turn0file0 L244-L258

9. FastAPI Backend Implementation

9.1 main.py —Basic Syntax

```
from fastapi import FastAPI
from routes import router

app = FastAPI(title="LegalEase API")

@app.get("/")
def root():
    return {"status": "LegalEase API is running"}

app.include_router(router)
```

9.2 routes.py —Request Model and Endpoint

```
from fastapi import APIRouter
from pydantic import BaseModel
from ai_core.gemini_generator import GeminiDocumentGenerator

router = APIRouter()
generator = GeminiDocumentGenerator()

class DocumentRequest(BaseModel):
    document_type: str
    parties: str
    terms: str
    dates: str

@router.post("/generate")
def generate_document(request: DocumentRequest):
    text = generator.generate_document(
        request.document_type,
        request.parties,
        request.terms,
        request.dates
    )
    return {"document": text}
```

9.3 API Syntax

POST /generate is the main document-generation endpoint. The request carries document_type, parties, terms and dates; the response returns generated document text. This follows the source project's API logic. filecite turn0file0 L181-L207

10. Gemini AI Integration

10.1 AI Generation Concept

The AI core receives structured user information and converts it into a prompt suitable for document generation. The supplied project identifies Gemini 1.5 Pro as its selected model and describes a GeminiDocumentGenerator component.

10.2 Illustrative Generator Syntax

```
import os
import google.generativeai as genai
from dotenv import load_dotenv

load_dotenv()
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

class GeminiDocumentGenerator:
    def generate_document(self, document_type, parties, terms, dates):
        prompt = f"""
        Create a structured draft for:
        Document Type: {document_type}
        Parties: {parties}
        Terms: {terms}
        Effective Date: {dates}

        Use clear sections and formal language.
        """
        model = genai.GenerativeModel("gemini-1.5-pro")
        response = model.generate_content(prompt)
        return response.text
```

10.3 Prompt Construction

Identify the requested document type.

Include parties and their roles.

Include terms and conditions.

Include the effective date.

Request structured sections suitable for later formatting.

Return text that can be reviewed before export.

The source describes structured prompting and the use of generate_content through

11. Streamlit Frontend

11.1 Basic Streamlit Syntax

```
import streamlit as st
import requests

st.set_page_config(page_title="LegalEase")

st.title("LegalEase")
document_type = st.text_input("Document Type")
parties = st.text_area("Parties")
terms = st.text_area("Terms")
dates = st.text_input("Effective Date")

if st.button("Generate Document"):
    payload = {
        "document_type": document_type,
        "parties": parties,
        "terms": terms,
        "dates": dates
    }
    response = requests.post(
        "http://127.0.0.1:8000/generate",
        json=payload
    )
    result = response.json()["document"]
    st.text_area("Generated Document", result, height=400)
```

11.2 UI Elements

Text input for document type and effective date.

Text area for parties and terms.

Generate Document button.

Editable generated-text area.

Download buttons for TXT, DOCX and PDF.

Styled HTML preview.

The source specifically describes Streamlit inputs, a three-column logo layout, document generation, HTML preview, editing and multi-format downloads. filecite turn0file0 L215-L242

12. Document Formatting & Export

12.1 DOCX Generation

```
from docx import Document

def format_docx(text, doc_type):
    document = Document()
    document.add_heading(doc_type, level=0)

    for paragraph in text.split("\n"):
        document.add_paragraph(paragraph)

    return document
```

12.2 PDF Generation

```
from fpdf import FPDF

def format_pdf(text, doc_type):
    pdf = FPDF()
    pdf.add_page()
    pdf.set_font("Arial", size=12)
    pdf.cell(0, 10, doc_type, ln=True, align="C")

    for line in text.split("\n"):
        pdf.multi_cell(0, 8, line)

    return pdf
```

12.3 Terms Processing

```
terms = "Payment within 30 days;Confidentiality;15-day termination notice"
items = [itemstrip() for item in terms.split(";") if itemstrip()]
```

The source specifies DOCX formatting with logo, headings, terms tables and footer, and PDF formatting with headers/footers and bullet-style terms. filecite turn0file0 L163-L179

13. User Operation / Project Work Flow



```
document_type = st.text_input("Document Type")
parties = st.text_area("Parties Involved")
terms = st.text_area("Terms & Conditions (Use semicolons for bullet points)")
dates = st.text_input("Effective Date")
```

Activity 4.2: Creating Dynamic Previews and Export Options

Step 1: Generate Document with Backend AI

- On clicking "Generate Document", a POST request is sent to the FastAPI backend:

```
response = requests.post("http://localhost:8000/generate", json={...})
```

- AI-generated content is sanitized and displayed dynamically:

```
generated_text = sanitize_text(response.json()["document"])
```

Step 2: HTML Preview Rendering

- The output is previewed in an elegant dark-themed scrollable card with semantic HTML:

```
styled_html = format_html_preview(generated_text)
st.markdown(f"<div style='...'>{styled_html}</div>", unsafe_allow_html=True)
```

Step 3: Editable Document Preview

- The user can click "Edit Document" and modify the generated content using a text area:

```
edited_text = st.text_area("Edit Document Below:", generated_text, height=300)
```

Step 4: Multi-Format Download Options

- Users can download the document as:
 - .txt (Plain Text)
 - .docx (Formatted Word Document with logo, headings, and table)
 - .pdf (Branded PDF with logo and footer)

```
st.download_button("Download as .TXT", data=generated_text, file_name=f"{document_type.replace(' ', '_').lower()}.txt", type="primary")
st.download_button("Download as .DOCX", data=format_docx(generated_text, document_type), file_name=f"{document_type.replace(' ', '_').lower()}.docx", type="primary")
st.download_button("Download as .PDF", data=format_pdf(generated_text, document_type), file_name=f"{document_type.replace(' ', '_').lower()}.pdf", type="primary")
```

Figure 3. Representative source-page view showing generation/preview/export workflow.

13.1 User Steps

Step 1: Open the LegalEase application.

Step 2: Select or enter the document type.

Step 3: Enter the parties involved.

Step 4: Enter terms and conditions using semicolons where required.

Step 5: Enter the effective date.

Step 6: Click Generate Document.

Step 7: Inspect the generated document preview.

Step 8: Click the edit option if changes are required.

Step 9: Choose TXT, DOCX or PDF output.

Step 10: Save/download the final draft for review.

The source examples include Agreement, Contract, NDA, Lease Agreement and Employment Offer Letter, with parties and terms entered through the interface. filecite turn0file0 L295-L325

14. Testing & Validation

14.1 Test Plan

Test Case	Input / Action	Expected Result
TC-01	Enter document type	Input accepted
TC-02	Enter parties	Party data accepted
TC-03	Enter semicolon-separated terms	Terms parsed as clauses
TC-04	Enter effective date	Date included in request
TC-05	Click Generate	Generated text returned
TC-06	Edit generated text	Updated text visible
TC-07	Download TXT	Plain-text file generated
TC-08	Download DOCX	Formatted Word document generated
TC-09	Download PDF	Formatted PDF generated
TC-10	Check branding	Logo/footer/layout appear as configured

14.2 Validation Checklist

FastAPI starts without errors.

Streamlit frontend loads correctly.

/generate accepts valid request data.

Gemini integration returns generated text.

Preview displays the result.

Editing does not remove required content unexpectedly.

TXT, DOCX and PDF exports work.

Terms, headings and branding remain readable.

Sensitive API keys are kept outside source code.

The supplied deployment section instructs testing different input combinations and checking preview, editing, downloads, layout, bullet formatting and terms embedding.

filecite turn0file0 L278-L293

15. Deployment, Conclusion & References

15.1 Local Deployment

```
# Start FastAPI
uvicorn main:app --reload

# Start Streamlit in another terminal
streamlit run app.py
```

The supplied project describes local deployment using a virtual environment, requirements.txt, Uvicorn for the FastAPI backend and Streamlit for the frontend. It also lists future deployment options such as Render, Railway, Deta, Streamlit Community Cloud, Fly.io or a VPS.

filecite turn0file0 L260-L289

15.2 Advantages

- Simple structured workflow for document drafting.
- AI-assisted generation reduces repetitive drafting work.
- Editable preview before export.
- Multiple output formats.
- Branding and consistent formatting support.
- Modular frontend/backend/AI architecture.

15.3 Limitations & Future Enhancements

- AI-generated legal text requires human review.
- Jurisdiction-specific legal validation is outside the supplied project specification.
- Future work may add deeper contract analysis, legal-database integration and personalized recommendations.
- Additional validation, authentication, audit logging and stronger privacy controls could be considered for a production system.

15.4 Conclusion

LegalEase demonstrates how a modern Python application can combine a Streamlit interface, FastAPI backend, Gemini-based generation and document-formatting utilities into one workflow. The project focuses on making structured legal-document drafting more accessible while providing preview, editing and export capabilities. Its modular architecture also provides a foundation for future improvements and deployment.

15.5 Source Reference

Primary source: supplied PDF, "LegalEase: AI-Powered Legal Document Generator," 25 pages.

The report preserves the terminology and workflow of the supplied material and expands it into an academic project-report format. filecite turn0file0 L2-L14
End of Project Report

